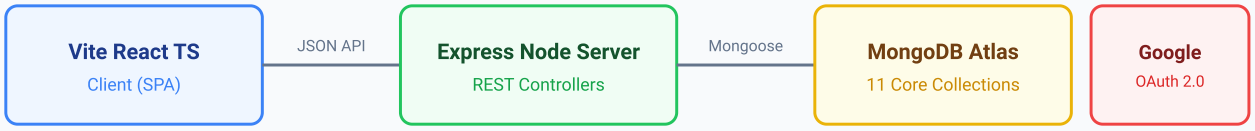
Auto Present V1

MongoDB + Mongoose Blueprint

Decoupled Architecture • Vite React TS Frontend • Node Express API • Google Auth

1. SYSTEM ARCHITECTURE & DATA FLOW



2. IDENTITY & ACADEMIC HIERARCHY SCHEMAS

1. users

Auth & Identity

Field	Type	Notes
_id	ObjectId	PK
email	String	**Unique, Indexed**
name	String	Full Name
role	Enum	Admin | CR | Student
googleId	String	OAuth Provider Key
avatar	String	Google Photo URL

2. academic_sessions

Session Control

Field	Type	Notes
_id	ObjectId	PK
name	String	**e.g. "2025-2026"**
year	Number	2026
isActive	Boolean	Only 1 Active at a time
createdBy	ObjectId	-> users (Admin)

3. departments

Dept Master

Field	Type	Notes
_id	ObjectId	PK
name	String	Computer Tech
code	String	**e.g. "66"**

4. classes

Class Grouping

Field	Type	Notes
_id	ObjectId	PK
sessionId	ObjectId	-> academic_sessions
departmentId	ObjectId	-> departments
shift	Enum	1st | 2nd Shift
semester	Number	1 to 8
group	String	e.g. "A" or "B"

3. STUDENT PROFILE & ENROLLMENT SCHEMAS

5. students

Permanent Profile

Field	Type	Notes
_id	ObjectId	PK
userId	ObjectId	-> users (Student)
roll	String	**Unique Board Roll**
registrationNo	String	**Unique Reg Number**
departmentId	ObjectId	-> departments

6. class_memberships

Session Enrollment

Field	Type	Notes
_id	ObjectId	PK
studentId	ObjectId	-> students
classId	ObjectId	-> classes
sessionId	ObjectId	-> academic_sessions

4. CR OPERATIONS & SCHEDULE SCHEMAS

7. cr_assignments

CR Permissions

Field	Type	Notes
_id	ObjectId	PK
userId	ObjectId	-> users (CR)
classId	ObjectId	-> classes
sessionId	ObjectId	-> academic_sessions
assignedBy	ObjectId	-> users (Admin)

8. class_routines

Time Schedule

Field	Type	Notes
_id	ObjectId	PK
classId	ObjectId	-> classes
dayOfWeek	String	Sunday .. Thursday
startTime	String	Format "08:00"
endTime	String	Format "08:45"
subject	String	Subject Code/Name

CR Scope & Validation Logic (V1)

Before allowing a CR to create a daily attendance window, the server performs a 3-way check: 1) Verify `cr_assignments` for `classId`. 2) Check `academic_sessions.isActive == true`. 3) Verify current server time falls inside `class_routines` start and end time.

5. CORE ATTENDANCE ENGINE & AUDIT SCHEMAS

9. class_sessions

Daily Attendance Window

Field	Type	Notes
_id	ObjectId	PK
classId	ObjectId	-> classes
crId	ObjectId	-> users (CR)
date	Date	ISO Date (YYYY-MM-DD)
status	Enum	Open | Closed
openedAt	Date	Timestamp
closedAt	Date	Auto/Manual Close

10. attendance_records

Student Attendance Entry

Field	Type	Notes
_id	ObjectId	PK
classSessionId	ObjectId	-> class_sessions
studentId	ObjectId	-> students
status	Enum	**Present | Late | Absent**
timestamp	Date	Entry Time

11. audit_logs

Security & Integrity Tracking

Field	Type	Purpose
_id	ObjectId	Unique log identifier
performedBy	ObjectId	User ID (Admin or CR) who executed action
action	String	CREATE_SESSION, TOGGLE_ATTENDANCE, ASSIGN_CR
details	String / JSON	Payload metadata or error details
timestamp	Date	Exact server timestamp

6. RECOMMENDED DATABASE INDEXES (COMPOUND & UNIQUE)

Collection	Index Definition	Primary Purpose
users	{ email: 1 } (Unique)	Instant login lookup & Google OAuth mapping
students	{ roll: 1 }, { registrationNo: 1 }	Prevent duplicate student record registration
attendance_records	{ classSessionId: 1, studentId: 1 } (Unique)	**Prevents double submission for same student**
class_memberships	{ studentId: 1, sessionId: 1 }	Fast Dashboard lookup for student profile

Production Checklist for Phase 1

✔ MongoDB Atlas cluster created & connection string configured in Node .env.
✔ Express backend running Mongoose Models with Schema validations.
✔ Google OAuth Client ID verified for React Vite frontend authentication flow.

7. REST API ENDPOINTS & ROUTING MATRIX

Base API URL: `/api/v1` • Authentication Header: `Authorization: Bearer <JWT_TOKEN>`

Admin Endpoints (/api/v1/admin)

Admin Role Only

Method	Endpoint	Purpose	Payload / Params
POST	`/sessions`	Create academic session	`{ name, year }`
PATCH	`/sessions/:id/active`	Set active academic session	Param: `id`
GET	`/sessions`	List all academic sessions	None
POST	`/departments`	Create department	`{ name, code }`
POST	`/classes`	Setup class for session & dept	`{ sessionId, departmentId, shift, semester, group }`
POST	`/cr-assignments`	Assign CR to class	`{ userId, classId, sessionId }`
POST	`/students`	Create student & enrollment	`{ name, email, roll, registrationNo, classId }`

CR Endpoints (/api/v1/cr)

CR Scope Restricted

Method	Endpoint	Purpose	Payload / Params
GET	`/assigned-classes`	Get list of CR's permitted classes	From Auth Token
GET	`/classes/:classId/students`	Get enrolled students in class	Param: `classId`
POST	`/class-sessions`	Open daily attendance window	`{ classId, date }`
POST	`/attendance`	Submit/Update student attendance	`{ classSessionId, records: [{ studentId, status }] }`
PATCH	`/class-sessions/:id/close`	Close attendance window	Param: `id`

Student Endpoints (/api/v1/student)

Student Self Read Only

Method	Endpoint	Purpose	Payload / Params
GET	`/profile`	Get logged-in profile & class info	From Auth Token
GET	`/attendance/history`	Get personal attendance history	Query: `?startDate=&endDate=`
GET	`/attendance/summary`	Get total attendance percentage	From Auth Token

Middleware Execution Flow

`Request` → `express.json() / CORS` → `authMiddleware` (Verify Google JWT) → `roleMiddleware` (Check Admin/CR/Student) → `crAccessMiddleware` (Check Class Ownership & Time Window) → `Controller`